

ATS Resume Scoring Assistant

Design Document

Document Version: 1.0

Last Updated: January 2024

Project: ATS Resume Scoring Assistant

ATS Resume Scoring Assistant - Design Document

1. Executive Summary
2. System Overview
3. Architecture
4. Technology Stack
5. Core Components
6. Multi-Agent Scoring System
7. Data Models
8. API Design
9. Frontend Architecture
10. Database Design
11. Security Architecture
12. Performance Considerations
13. Deployment Architecture
14. Future Enhancements

Executive Summary

The ATS Resume Scoring Assistant is a comprehensive web application designed to automate and enhance the resume screening process for recruiters and hiring managers. The system leverages advanced AI technologies including multi-agent architecture, semantic similarity matching, and machine learning to provide intelligent resume-to-job matching with detailed scoring and analysis.

Key Features

- **Multi-Agent Scoring System:** Five specialized AI agents analyze different aspects of resume-job matching

- **Intelligent Document Processing:** PDF/DOCX resume parsing with text extraction
- **Semantic Similarity Analysis:** Vector-based matching using ChromaDB and sentence transformers
- **Comprehensive Scoring:** Weighted scoring across keywords, skills, experience, education, and semantic similarity
- **Modern Web Interface:** React-based frontend with Swiss design principles
- **Secure Authentication:** JWT-based user authentication and authorization

System Overview

Purpose

The system addresses the challenge of manual resume screening by providing automated, intelligent matching between resumes and job descriptions. It helps recruiters identify the best candidates quickly and objectively.

Target Users

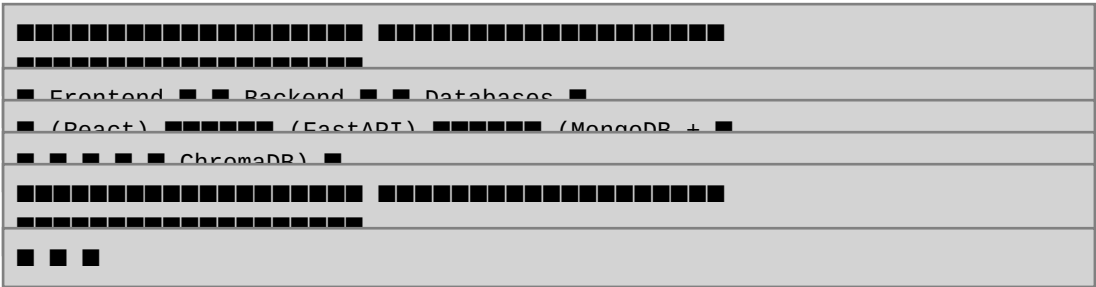
- **Primary:** Recruiters and HR professionals
- **Secondary:** Hiring managers and talent acquisition teams
- **Tertiary:** Job seekers (for self-assessment)

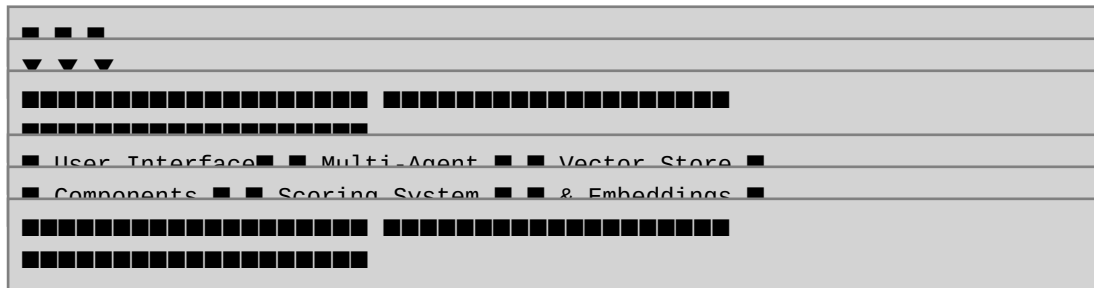
Core Workflow

1. User uploads resume files (PDF/DOCX)
2. User creates or uploads job descriptions
3. System processes documents and extracts structured data
4. Multi-agent system analyzes resume-job compatibility
5. Results displayed with detailed scoring breakdown
6. Users can review, compare, and make informed decisions

Architecture

High-Level Architecture





System Components

Frontend Layer

- **React 18** with TypeScript
- **Vite** for build tooling
- **TailwindCSS** for styling
- **React Router** for navigation
- **Axios** for API communication

Backend Layer

- **FastAPI** for REST API
- **Pydantic** for data validation
- **Uvicorn** as ASGI server
- **Multi-Agent Architecture** for scoring

Data Layer

- **MongoDB** for structured data
- **ChromaDB** for vector storage
- **In-Memory Database** for development
- **File System** for document storage

Technology Stack

Frontend Technologies

Backend Technologies

Database Technologies

Core Components

1. Authentication System

Technology	Version	Purpose
-----	-----	-----
React	18.x	UI Framework
TypeScript	5.x	Type Safety
Vite	7.1.4	Build Tool
TailwindCSS	3.x	Styling
React Router	6.x	Routing
Axios	1.x	HTTP Client
Lucide React	0.x	Icons
Technology	Version	Purpose
-----	-----	-----
Python	3.12+	Runtime
FastAPI	0.104+	Web Framework
Pydantic	2.x	Data Validation
Uvicorn	0.24+	ASGI Server
PyPDF2	3.x	PDF Processing
python-docx	0.8.x	DOCX Processing
sentence-transformers	2.x	Embeddings
ChromaDB	0.4.x	Vector Database
JWT	1.7.x	Authentication
bcrypt	4.x	Password Hashing
Technology	Purpose	Usage
-----	-----	-----
MongoDB	Primary Database	User data, resumes, jobs, results
ChromaDB	Vector Database	Embeddings, similarity search
In-Memory	Development	Local development and testing

- **JWT-based authentication** with secure token management
- **Password hashing** using bcrypt
- **User registration and login** with email validation
- **Protected routes** and API endpoints
- **Session management** with token refresh

2. Document Processing Engine

- **PDF parsing** using PyPDF2 for text extraction
- **DOCX processing** using python-docx
- **Text normalization** and cleaning
- **Metadata extraction** (file size, hash, etc.)
- **Error handling** for corrupted or unsupported files

3. Job Description Management

- Structured job creation with required fields
- Skills and requirements parsing
- Experience level categorization
- Company and location information
- Salary range specification

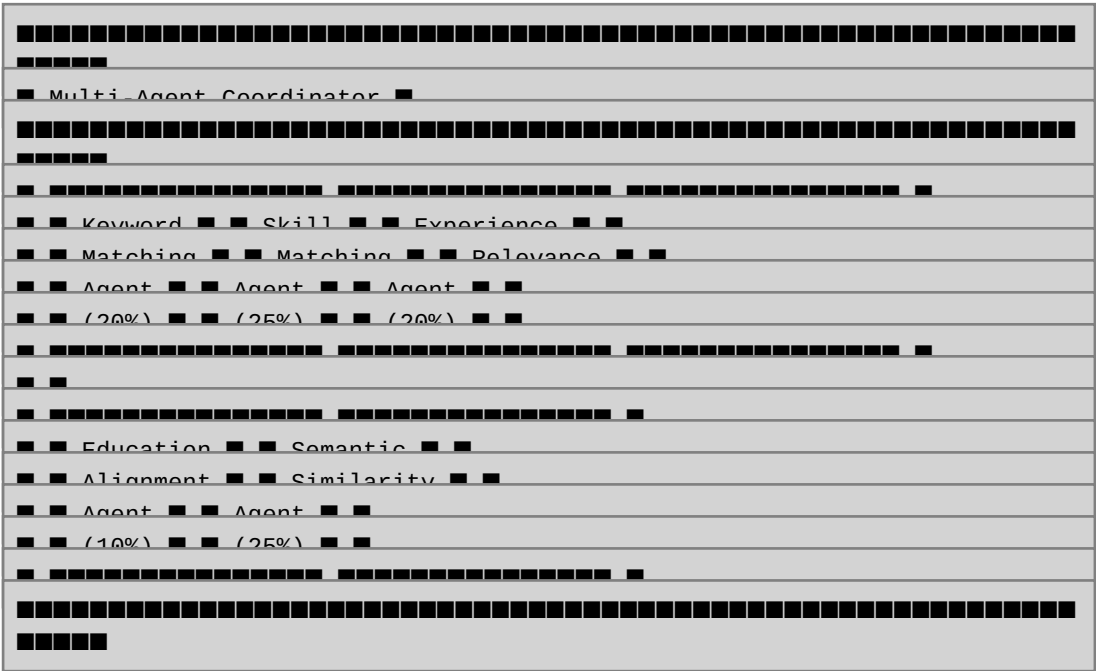
4. Resume Management System

- File upload with drag-and-drop interface
- Resume parsing and text extraction
- Skills extraction using AI
- Experience calculation from text analysis
- Education parsing and categorization

Multi-Agent Scoring System

Architecture Overview

The multi-agent system consists of five specialized agents that analyze different aspects of resume-job matching:



Agent Specifications

1. Keyword Matching Agent (Weight: 20%)

- Purpose: Identifies exact keyword matches between resume and job description

- **Method:** Text analysis and frequency counting
- **Output:** Keyword match percentage and missing keywords
- **Confidence:** High for exact matches, medium for variations

2. Skill Matching Agent (Weight: 25%)

- **Purpose:** Matches technical and soft skills using taxonomy
- **Method:** Skill extraction and normalization using LLM
- **Output:** Skills match percentage, matched skills, missing skills
- **Confidence:** High for technical skills, medium for soft skills

3. Experience Relevance Agent (Weight: 20%)

- **Purpose:** Analyzes years of experience and seniority level
- **Method:** Experience extraction and level comparison
- **Output:** Experience score, years match, level alignment
- **Confidence:** High for clear experience indicators

4. Education Alignment Agent (Weight: 10%)

- **Purpose:** Matches educational background with job requirements
- **Method:** Degree level mapping and field relevance
- **Output:** Education score, degree match, field relevance
- **Confidence:** High for degree requirements, medium for field relevance

5. Semantic Similarity Agent (Weight: 25%)

- **Purpose:** Analyzes semantic similarity using embeddings
- **Method:** Vector similarity using sentence transformers
- **Output:** Semantic similarity score and confidence
- **Confidence:** Medium to high based on embedding quality

Scoring Algorithm

total_score = (
keyword_score * 0.20 +
skill_score * 0.25 +
experience_score * 0.20 +
education_score * 0.10 +
semantic_score * 0.25
)

Parallel Processing

- All agents run **asynchronously** for optimal performance
- **Error handling** ensures system resilience
- **Confidence weighting** for result reliability
- **Evidence collection** for transparency

Data Models

User Model

class UserResponse(BaseModel):
id: str
email: EmailStr
full_name: str
created_at: datetime
updated_at: datetime

Resume Model

class ResumeResponse(BaseModel):
id: str
user_id: str
title: str
filename: str
file_path: str
file_size: int
text_content: str
file_hash: str
text_hash: str
skills: List[str]
experience_years: int
education: str
created_at: datetime
updated_at: datetime

Job Model

class JobResponse(BaseModel):
id: str
user_id: str
title: str
company: str
description: str
requirements: List[str]
skills: List[str]
experience_level: str
location: str
salary_range: str
created_at: datetime
updated_at: datetime

Scoring Result Model

class ScoringBreakdown(BaseModel):
keyword_match: Dict[str, Any]
skills_alignment: Dict[str, Any]
experience_relevance: Dict[str, Any]
education_alignment: Dict[str, Any]
semantic_similarity: Dict[str, Any]
total_score: float
match_percentage: float

skills_match: List[Str]
missing_skills: List[Str]
agent_results: Dict[Str, AgentResult]
confidence: float
timestamp: datetime

API Design

Authentication Endpoints

POST /api/auth/signup # User registration
POST /api/auth/login # User login
GET /api/auth/me # Get current user

Resume Endpoints

POST /api/resumes/upload # Upload resume file
GET /api/resumes/ # Get user's resumes
GET /api/resumes/{id} # Get specific resume
PUT /api/resumes/{id} # Update resume
DELETE /api/resumes/{id} # Delete resume

Job Endpoints

POST /api/jobs/ # Create job description
GET /api/jobs/ # Get user's jobs
GET /api/jobs/{id} # Get specific job
PUT /api/jobs/{id} # Update job
DELETE /api/jobs/{id} # Delete job

Scoring Endpoints

POST /api/scoring/score/{job_id} # Score resumes for job
GET /api/scoring/results/{job_id} # Get scoring results
GET /api/scoring/history # Get scoring history

API Response Format

{
"success": true
"data": { } }
"message": "Operation completed successfully"
"timestamp": "2024-01-01T00:00:00Z"
}

Error Response Format

{
"success": false
"error": {
"code": "VALIDATION_ERROR"
"message": "Invalid input data"
"details": { } }
}
"timestamp": "2024-01-01T00:00:00Z"


```
}
```

Frontend Architecture

Component Structure

```
src/  
  ■■■ components/ # Reusable UI components  
  ■ ■■■ Navbar.tsx # Navigation component  
  ■ ■■■ FileUpload.tsx # File upload component  
  ■ ■■■ JobForm.tsx # Job creation form  
  ■ ■■■ ScoreCard.tsx # Scoring results display  
  ■■■ pages/ # Page components  
  ■ ■■■ Login.tsx # Login page  
  ■ ■■■ Signup.tsx # Registration page  
  ■ ■■■ Dashboard.tsx # Main dashboard  
  ■ ■■■ ResumeUpload.tsx # Resume upload page  
  ■ ■■■ JobDescription.tsx # Job creation page  
  ■ ■■■ ScoringResults.tsx # Results display page  
  ■■■ hooks/ # Custom React hooks  
  ■ ■■■ useAuth.tsx # Authentication hook  
  ■■■ lib/ # Utilities and API client  
  ■ ■■■ api.ts # API client configuration  
  ■■■ App.tsx # Main application component
```

State Management

- **React Context** for global state (authentication)
- **Local State** with `useState` for component state
- **Custom Hooks** for reusable logic
- **API Integration** with Axios interceptors

Routing Structure

```
/ # Landing page  
/login # User login  
/signup # User registration  
/dashboard # Main dashboard (protected)  
/upload # Resume upload (protected)  
/jobs # Job management (protected)  
/results # Scoring results (protected)
```

Design System

- **Swiss Design Principles:** Clean, minimal, functional
- **Color Palette:** Blue primary, neutral grays, semantic colors
- **Typography:** Inter font family with clear hierarchy
- **Spacing:** Consistent 8px grid system
- **Components:** Reusable, accessible UI components

Database Design

MongoDB Collections

Users Collection

{
id: ObjectId
email: String (unique)
full name: String
hashed password: String
created at: Date
updated at: Date
}

Resumes Collection

{
id: ObjectId
user id: ObjectId
title: String
filename: String
file path: String
file size: Number
text content: String
file hash: String
text hash: String
skills: [String]
experience years: Number
education: String
created at: Date
updated at: Date
}

Jobs Collection

{
id: ObjectId
user id: ObjectId
title: String
company: String
description: String
requirements: [String]
skills: [String]
experience level: String
location: String
salary range: String
created at: Date
updated at: Date
}

Scoring Results Collection

{
_id: ObjectId,

user_id: ObjectId
job_id: ObjectId
resume_id: ObjectId
scoring_breakdown: Object
total_score: Number
match_percentage: Number
confidence: Number
created_at: Date
}

ChromaDB Collections

- **Resume Embeddings:** Vector representations of resume content
- **Job Embeddings:** Vector representations of job descriptions
- **Metadata:** Associated document information and IDs

Security Architecture

Authentication & Authorization

- **JWT Tokens:** Secure, stateless authentication
- **Password Hashing:** bcrypt with salt rounds
- **Token Expiration:** Configurable token lifetime
- **Route Protection:** Middleware-based access control

Data Security

- **Input Validation:** Pydantic models for data validation
- **File Upload Security:** File type and size validation
- **SQL Injection Prevention:** Parameterized queries
- **XSS Protection:** Input sanitization and output encoding

API Security

- **CORS Configuration:** Controlled cross-origin requests
- **Rate Limiting:** Request throttling (future enhancement)
- **HTTPS Enforcement:** SSL/TLS encryption
- **API Key Management:** Secure key storage (future enhancement)

File Security

- **Upload Validation:** File type and size restrictions
- **Virus Scanning:** File content validation (future enhancement)
- **Secure Storage:** Isolated file storage with access controls
- **File Hashing:** Integrity verification

Performance Considerations

Backend Performance

- **Async Processing:** FastAPI with async/await
- **Parallel Agent Execution:** Concurrent scoring agents
- **Database Indexing:** Optimized MongoDB queries
- **Caching Strategy:** Redis for session and result caching (future)
- **Connection Pooling:** Efficient database connections

Frontend Performance

- **Code Splitting:** Route-based lazy loading
- **Bundle Optimization:** Vite build optimization
- **Image Optimization:** Compressed assets
- **Caching:** Browser caching for static assets
- **Virtual Scrolling:** Large list performance (future)

Database Performance

- **Indexing Strategy:** Compound indexes for common queries
- **Query Optimization:** Efficient aggregation pipelines
- **Vector Search:** Optimized ChromaDB queries
- **Connection Management:** Connection pooling and timeouts

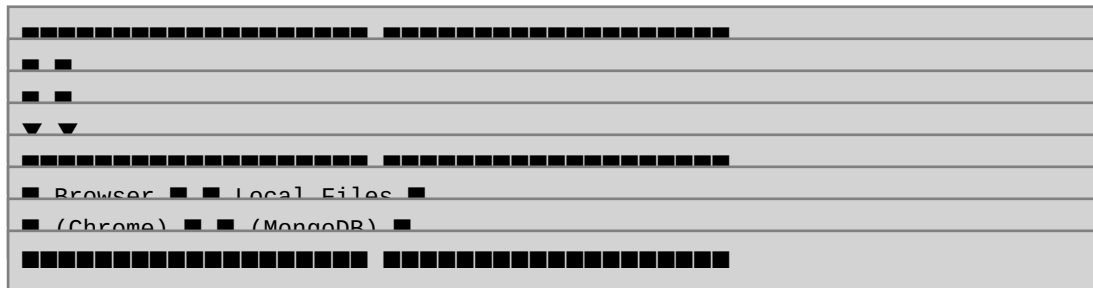
Scalability Considerations

- **Horizontal Scaling:** Stateless application design
- **Load Balancing:** Multiple backend instances
- **Database Sharding:** MongoDB sharding strategy
- **CDN Integration:** Static asset delivery
- **Microservices:** Service decomposition (future)

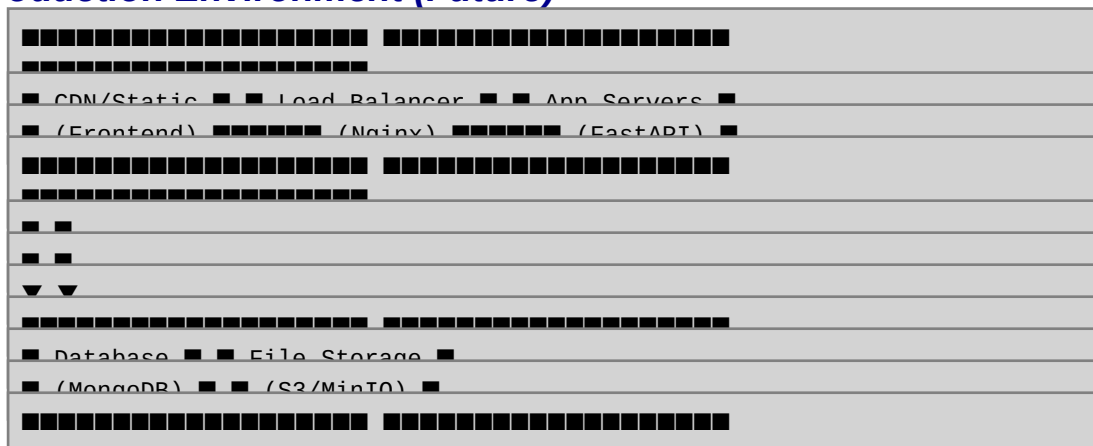
Deployment Architecture

Development Environment

[illegible]



Production Environment (Future)



Containerization

- **Docker:** Application containerization
- **Docker Compose:** Multi-service orchestration
- **Kubernetes:** Production orchestration (future)
- **Health Checks:** Application monitoring

Environment Configuration

- **Environment Variables:** Configuration management
- **Secrets Management:** Secure credential storage
- **Configuration Files:** Environment-specific settings
- **Feature Flags:** Runtime feature toggling

Future Enhancements

Short-term Enhancements (3-6 months)

1. Advanced Analytics Dashboard

- Scoring trends and patterns
- Performance metrics
- User activity analytics

2. Enhanced Document Processing

- OCR for scanned documents

- Multiple file format support
- Document template recognition

3. Improved AI Models

- Fine-tuned embedding models
- Custom skill taxonomy
- Industry-specific scoring

Medium-term Enhancements (6-12 months)

1. Multi-language Support

- Internationalization (i18n)
- Multi-language document processing
- Localized scoring algorithms

2. Advanced Matching Features

- Fuzzy skill matching
- Context-aware scoring
- Industry-specific weights

3. Integration Capabilities

- ATS system integration
- HRIS connectivity
- API webhooks

Long-term Enhancements (12+ months)

1. Machine Learning Pipeline

- Continuous learning from user feedback
- Automated model retraining
- Performance optimization

2. Enterprise Features

- Multi-tenant architecture
- Advanced user management
- Compliance and audit trails

3. Mobile Application

- Native mobile apps
- Offline capabilities
- Push notifications

Technical Debt and Improvements

1. Testing Coverage

- Unit test coverage > 90%
- Integration testing

- End-to-end testing

2. Monitoring and Observability

- Application performance monitoring
- Error tracking and alerting
- Business metrics dashboard

3. Security Enhancements

- Penetration testing
- Security audit
- Compliance certification

Conclusion

The ATS Resume Scoring Assistant represents a comprehensive solution for automated resume screening, combining modern web technologies with advanced AI capabilities. The multi-agent architecture provides robust, transparent, and accurate scoring while maintaining system flexibility and extensibility.

The system's design emphasizes:

- **Scalability:** Architecture supports growth and expansion
- **Maintainability:** Clean code structure and documentation
- **Performance:** Optimized for speed and efficiency
- **Security:** Comprehensive security measures
- **User Experience:** Intuitive and responsive interface

This design document serves as a blueprint for development, deployment, and future enhancements of the ATS Resume Scoring Assistant system.

Document Version: 1.0

Last Updated: January 2024

Author: AI Assistant

Project: ATS Resume Scoring Assistant